

Building a gRPC Key-Value Store

<https://gemini.google.com/app/b653aa32c15fe2a0>

User prompt: what are the next steps?

Response: Based on your progress in setting up and verifying your gRPC environment , you are now ready to step away from the gRPC installation directory and start constructing your actual custom key-value store project.

The concrete **next steps** required to fulfill **Phase 1: Single-Node Core & Networking** are outlined below:

1. Set up your Project Directory

First, organize your project structure by creating a dedicated folder and the necessary source files:

```
kv-store/
├── CMakeLists.txt
├── proto/
│   └── kv.proto
└── src/
    ├── storage_engine.h
    ├── storage_engine.cpp
    ├── server.cpp
    └── client.cpp
```

2. Define the gRPC Service Contract (proto/kv.proto)

Write the foundational Protocol Buffers file to define the API contract for the data storage operations. Keep it simple without clustering or replication options for now:

```
syntax = "proto3";
package kvstore;

service KeyValueStore {
  rpc Put(PutRequest) returns (PutResponse);
  rpc Get(GetRequest) returns (GetResponse);
  rpc Delete(DeleteRequest) returns (DeleteResponse);
}

message PutRequest {
  string key = 1;
  bytes value = 2; // Content-agnostic to support binary blobs
}

message PutResponse {
  bool success = 1;
}

message GetRequest {
  string key = 1;
}

message GetResponse {
  bool found = 1;
  bytes value = 2;
}

message DeleteRequest {
  string key = 1;
}

message DeleteResponse {
  bool success = 1;
}
```

3. Populate the Custom CMakeLists.txt

Place the highly optimized CMake script provided previously into your root `kv-store/` folder. It enforces the modern C++17 standard , directs CMake to locate the locally compiled gRPC and Protobuf installations inside your `~/local` prefix , and automates code generation for the network stubs from `kv.proto`.

4. Implement the Storage Engine with Move Semantics (src/)

Implement the thread-safe in-memory map protected by a reader-writer lock (`std::shared_mutex`). To make the database production-grade and performant for large binary blobs, use C++ move semantics to prevent expensive deep memory copies.

- `src/storage_engine.h`:

```
#pragma once
#include <string>
#include <unordered_map>
#include <shared_mutex>
#include <optional>
```

```

class StorageEngine {
public:
    void Put(std::string key, std::string value);
    std::optional<std::string> Get(const std::string& key) const;
    bool Delete(const std::string& key);
private:
    mutable std::shared_mutex mutex_;
    std::unordered_map<std::string, std::string> data_;
};

• src/storage_engine.cpp:

#include "storage_engine.h"

void StorageEngine::Put(std::string key, std::string value) {
    std::unique_lock lock(mutex_); // Exclusive lock for write operations
    data_[std::move(key)] = std::move(value); // Stolen directly from network buffer without copy
}

std::optional<std::string> StorageEngine::Get(const std::string& key) const {
    std::shared_lock lock(mutex_); // Shared lock allows multiple concurrent readers
    auto it = data_.find(key);
    if (it == data_.end()) return std::nullopt;
    return it->second;
}

bool StorageEngine::Delete(const std::string& key) {
    std::unique_lock lock(mutex_);
    return data_.erase(key) > 0;
}

```

5. Write the Server and Client Implementations (src/server.cpp & src/client.cpp)

- In **server.cpp**: Inherit from the generated `kvstore::KeyValueStore::Service` skeleton and map the RPC methods directly into your `StorageEngine` object. Ensure that the server takes a port argument from the CLI so you can effortlessly launch multiple local processes later on.
- In **client.cpp**: Establish an insecure network channel targeting `localhost:50051`, wrap it inside a gRPC `Stub`, and call `Put` and `Get` to execute baseline validation.

6. Compile and Sanity Check

Move into your project root directory and build using your isolated construction zone pattern:

```

cd ~/Developer/kv-store
mkdir -p build && cd build
cmake -DCMAKE_PREFIX_PATH=$HOME/.local ..
make -j$(sysctl -n hw.ncpu)

```

Launch `./kv_server` in one terminal window and run `./kv_client` in another to confirm that keys are being set and retrieved properly. As a final check before completing Phase 1, stress test your binaries concurrently under ThreadSanitizer (`-fsanitize=thread`) to guarantee your implementation is completely free of data races.

Once these components are validated, you will have a solid single-node storage baseline and can immediately progress to **Phase 2: Sharding via Consistent Hashing**. Let me know which specific component of Phase 1 you would like to write out or inspect next!

User prompt: I have few doubts. Let's consider that hello world example. The `./greeter_server` and `./greeter_client` commands I ran in separate terminals, are these executables? And also, are these create when I do `cmake` and then `make -j` in the `cmake/build` folder?

Response: Yes, you are exactly right on both counts! Let's clear up these doubts by breaking down exactly what happens under the hood during that process.

1. Are `./greeter_server` and `./greeter_client` executables?

Yes, they are standalone executable binaries. When you compile C++ code, the compiler transforms your human-readable text code (`.cc` files) into machine code (binary digits) that your computer's CPU can understand directly.

- The prefix `./` in your terminal literally tells your command-line shell: "Look in the current active directory (`.`) and execute the binary file named `greeter_server` or `greeter_client`."
- Because they are independent executables, you are able to run them in completely separate terminal windows as distinct operating system processes that communicate across network ports.

2. Are they created when running `cmake` and `make -j`?

Yes, they are generated exactly during this two-step build pipeline. However, `cmake` and `make` perform two fundamentally different jobs to create them:

Step A: The `cmake` Command (The Architect)

When you run a command like `cmake -DCMAKE_PREFIX_PATH=$MY_INSTALL_DIR ../..`, you are **not** compiling any code yet.

- Instead, CMake reads the `CMakeLists.txt` file in the project directory.
- Inside that file, commands like `add_executable(greeter_server greeter_server.cc)` tell CMake: "Hey, I want to eventually create an executable binary out of this source file."
- CMake checks your system for compilers, locates where your gRPC/Protobuf libraries are hiding, and generates the actual low-level platform-specific build instructions—usually a file called a **Makefile**.

Step B: The `make -j` Command (The Builder)

When you type `make -j 4`, you are telling the build system to actually execute the Makefile instructions generated by CMake.

- This command invokes your system's underlying C++ compiler (like `g++` or `clang++`).
- The compiler takes the source code files, compiles them, links them with the compiled gRPC and Protobuf network libraries, and outputs the final runnable machine-code binaries (`greeter_server` and `greeter_client`) directly into your current `cmake/build` folder.
- The `-j 4` flag simply tells `make` to use 4 CPU cores in parallel to speed up this compilation work.

Once `make` finishes successfully, the binaries pop up in your build folder, and they are ready to be run with `./!`

User prompt: In this `CMakeLists.txt` file, which lines tell the protobuf and grpc to first compile and create the `.cc` and `.h` files, and which lines tell the `cpp` compiler to compile these `cpp` files along with the other `cpp` files present, and then create binaries?? # Copyright 2018 gRPC authors. # Licensed under the Apache License, Version 2.0 (the "License"); # you may not use this file except in compliance with the License. # You may obtain a copy of the License at # http://www.apache.org/licenses/LICENSE-2.0 # Unless required by applicable law or agreed to in writing, software # distributed under the License is distributed on an "AS IS" BASIS, # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. # See the License for the specific language governing permissions and # limitations under the License. # # cmake build file for C++ helloworld example. # Assumes protobuf and gRPC have been installed using cmake. # See cmake_externalproject/CMakeLists.txt for all-in-one cmake build # that automatically builds all the dependencies before building helloworld. cmake_minimum_required(VERSION 3.16) project(HelloWorld C CXX) include(..cmake/common.cmake) # Proto file get_filename_component(hw_proto "../protos/helloworld.proto" ABSOLUTE) get_filename_component(hw_proto_path "\${hw_proto}" PATH) # Generated sources set(hw_proto_srcs "\${CMAKE_CURRENT_BINARY_DIR}/helloworld.pb.cc") set(hw_proto_hdrs "\${CMAKE_CURRENT_BINARY_DIR}/helloworld.pb.h") set(hw_grpc_srcs "\${CMAKE_CURRENT_BINARY_DIR}/helloworld.grpc.pb.cc") set(hw_grpc_hdrs "\${CMAKE_CURRENT_BINARY_DIR}/helloworld.grpc.pb.h") add_custom_command(OUTPUT "\${hw_proto_srcs}" "\${hw_proto_hdrs}" "\${hw_grpc_srcs}" "\${hw_grpc_hdrs}" COMMAND \${_PROTOBUF_PROTOC} ARGS --grpc_out "\${CMAKE_CURRENT_BINARY_DIR}" --cpp_out "\${CMAKE_CURRENT_BINARY_DIR}" -I "\${hw_proto_path}" --plugin=protoc-gen-grpc="\${_GRPC_CPP_PLUGIN_EXECUTABLE}" "\${hw_proto}" DEPENDS "\${hw_proto}") # Include generated *.pb.h files include_directories("\${CMAKE_CURRENT_BINARY_DIR}") # hw_grpc_proto add_library(hw_grpc_proto \${hw_grpc_srcs} \${hw_grpc_hdrs} \${hw_proto_srcs} \${hw_proto_hdrs}) target_link_libraries(hw_grpc_proto absl::check \${_REFLECTION} \${_GRPC_GRPCPP} \${_PROTOBUF_LIBPROTOBUF}) # Targets greeter_async_(client|server) foreach(_target greeter_client greeter_server greeter_callback_client greeter_callback_server greeter_async_client greeter_async_client2 greeter_async_server) add_executable(\${_target} "\${_target}.cc") target_link_libraries(\${_target} hw_grpc_proto absl::check absl::flags absl::flags_parse absl::log absl::log_initialize \${_REFLECTION} \${_GRPC_GRPCPP} \${_PROTOBUF_LIBPROTOBUF}) endforeach()

Response: This `CMakeLists.txt` file uses a very structured, three-step pipeline to handle code generation, library compilation, and executable linking.

Here is exactly which lines perform each task:

Part 1: Generating the `.cc` and `.h` files from the `.proto` file

The code generation is driven by the `add_custom_command` block (Lines 31–41), but it relies on `add_library` (Lines 47–51) to actually trigger.

```
add_custom_command(
  OUTPUT "${hw_proto_srcs}" "${hw_proto_hdrs}" "${hw_grpc_srcs}" "${hw_grpc_hdrs}"
  COMMAND ${_PROTOBUF_PROTOC}
  ARGS --grpc_out "${CMAKE_CURRENT_BINARY_DIR}"
  --cpp_out "${CMAKE_CURRENT_BINARY_DIR}"
  -I "${hw_proto_path}"
  --plugin=protoc-gen-grpc="${_GRPC_CPP_PLUGIN_EXECUTABLE}"
  "${hw_proto}"
  DEPENDS "${hw_proto}")
```

- **What this does:** This explicitly tells CMake, "If someone asks for these output `.cc` and `.h` files, run the Protocol Buffer compiler (`${_PROTOBUF_PROTOC}`) along with the gRPC C++ plugin (`${_GRPC_CPP_PLUGIN_EXECUTABLE}`) to generate them from `helloworld.proto`."
- **The Catch:** In CMake, an `add_custom_command` is lazy. It won't actually run just because it is written down. It only runs if another target asks for its `OUTPUT` files.

That trigger happens right below it:

```
add_library(hw_grpc_proto
  ${hw_grpc_srcs}
  ${hw_grpc_hdrs}
  ${hw_proto_srcs}
  ${hw_proto_hdrs})
```

- **What this does:** This tells the C++ compiler to bundle those generated `.cc` and `.h` files into a reusable internal library target named

`hw_grpc_proto`. Because this library requires those files, it forces CMake to execute the `add_custom_command` above during the `make` phase to generate them first.

Part 2: Compiling the source files and creating the final binaries

The compilation of your human-written code and the final creation of the executables happens inside the **foreach loop** (Lines 60–75).

```
foreach(_target
  greeter_client greeter_server
  greeter_callback_client greeter_callback_server
  greeter_async_client greeter_async_client2 greeter_async_server)
  add_executable(${_target} "${_target}.cc")
  target_link_libraries(${_target}
    hw_grpc_proto
    absl::check
    ...
    ${_GRPC_GRPCPP}
    ${_PROTOBUF_LIBPROTOBUF})
endforeach()
```

This loop iterates through all the programs (like `greeter_client` and `greeter_server`) and runs two key operations on each:

1. `add_executable(${_target} "${_target}.cc")`
 - **What this does:** This tells the C++ compiler to look at your human-written file (e.g., `greeter_client.cc`) and compile it into an executable binary target.
2. `target_link_libraries(${_target} hw_grpc_proto ...)`
 - **What this does:** A client or server program cannot run on its own because its code references gRPC routines and the generated proto network stubs. This line tells the **Linker** to stitch everything together.
 - It takes the compiled `greeter_client.cc` machine code, links it with the `hw_grpc_proto` library (the compiled proto files from Part 1), and hooks it up to the core gRPC and Protobuf binary libraries (`${_GRPC_GRPCPP}` and `${_PROTOBUF_LIBPROTOBUF}`) installed on your computer.

Summary of the execution flow when you run `make`:

1. `make` sees `greeter_client` needs `hw_grpc_proto`.
2. `make` checks `hw_grpc_proto` and notices the `.pb.cc` files don't exist yet.
3. It triggers `add_custom_command`, running `protoc` to create the `.cc` and `.h` files.
4. The C++ compiler compiles those brand new `.pb.cc` files into the `hw_grpc_proto` library.
5. The C++ compiler compiles `greeter_client.cc`.
6. The Linker joins `greeter_client.o`, `hw_grpc_proto`, and system gRPC binaries to produce the final executable `./greeter_client`.

User prompt: "It takes the compiled `greeter_client.cc` machine code, links it with the `hw_grpc_proto` library (the compiled proto files from Part 1)..", here, how do compiled `.cc` files get linked to uncompiled `.cc` and `.h` files (that were created by `protoc` from `.proto` file)? is it possible?

Response: No, **it is absolutely not possible** to link compiled machine code to uncompiled source files (`.cc` and `.h`). You have caught a very subtle and common point of confusion in C++ build pipelines!

The linker only understands machine code (binary). It cannot read raw text files.

The confusion usually comes from how we describe the steps. When we say it links `greeter_client.cc` with the `hw_grpc_proto` library, **both of them have already been compiled into binary form** by the time the linker starts its job.

Here is the exact step-by-step timeline of how those uncompiled files get turned into machine code *before* they ever touch the linker:

Step 1: Code Generation (Raw Text → Raw Text)

The Protobuf compiler (`protoc`) runs first. It reads your `helloworld.proto` file and generates:

- `helloworld.pb.h` & `helloworld.pb.cc`
- `helloworld.grpc.pb.h` & `helloworld.grpc.pb.cc`

At this exact moment, these 4 files are **uncompiled, raw C++ text code**.

Step 2: Independent Compilation (Raw Text → Machine Code)

Next, the C++ compiler (`g++` or `clang++`) steps in. It compiles the files completely independently of each other into temporary binary files called **Object Files** (`.o`):

1. **Compiling the Proto Files:** The compiler takes the freshly generated `helloworld.pb.cc` and `helloworld.grpc.pb.cc` and compiles them into binary object files. CMake then glues these object files together into a static library file (like `libhw_grpc_proto.a`). **This library is now pure compiled machine code.**

2. **Compiling your Client Code:** The compiler takes your human-written `greeter_client.cc` and compiles it into `greeter_client.o`.

Wait, how does `greeter_client.cc` compile if it doesn't know what's inside the proto files yet? This is what the `.h` (header) files are for! Inside `greeter_client.cc`, you wrote `#include "helloworld.grpc.pb.h"`. The compiler reads that header file as a "blueprint" or a promise. The header says: "Hey compiler, I promise a function named `SayHello()` exists somewhere. Just trust me and compile `greeter_client.cc` for now." The compiler leaves a placeholder blank spot in `greeter_client.o` where that function should go.

Step 3: Linking (Machine Code + Machine Code → Executable)

Now—and only now—the **Linker** enters the room.

The linker looks at `greeter_client.o` (compiled) and sees the blank placeholder spots for the gRPC functions. It then looks inside the `libhw_grpc_proto.a` library (also compiled), finds the machine code for those exact gRPC functions, and physically stitches them into the blank spots.

The result of joining these two pre-compiled binary pieces together is your final executable: `./greeter_client`.

Summary

- **protoc** creates uncompiled text files (`.cc` / `.h`).
- **The Compiler** turns **both** your client code and those generated text files into separate binary chunks.
- **The Linker** connects those binary chunks together. Uncompiled files are never handled by the linker.

User prompt: I mean where in the `cmakelists.txt` code is written to compile the `.cc` and `.h` files created by `protoc`

Response: The exact line of code that tells the C++ compiler to compile the `.cc` files created by `protoc` is this one:

```
# hw_grpc_proto
add_library(hw_grpc_proto
    ${hw_grpc_srcs} # <--- Compiles helloworld.grpc.pb.cc
    ${hw_grpc_hdrs}
    ${hw_proto_srcs} # <--- Compiles helloworld.pb.cc
    ${hw_proto_hdrs})
```

How does this line tell the compiler to compile them?

In CMake, the `add_library(...)` command is the instruction that tells the build system: "Take these source files, compile them using the C++ compiler, and bundle them into a library."

Here is exactly how CMake translates those lines behind the scenes:

1. **Variables are Expanded:** Earlier in the file, you defined variables holding the paths to the files:
 - `${hw_proto_srcs}` expands to `helloworld.pb.cc`
 - `${hw_grpc_srcs}` expands to `helloworld.grpc.pb.cc`
2. **The Command is Read:** When CMake reads `add_library(hw_grpc_proto ...)`, it sees those `.cc` files in the list.
3. **Compiler Commands are Generated:** During the `make` step, your build system sees this library target and runs the C++ compiler (like `g++` or `clang++`) on those files:


```
g++ -c helloworld.pb.cc -o helloworld.pb.cc.o
g++ -c helloworld.grpc.pb.cc -o helloworld.grpc.pb.cc.o
```

(The `-c` flag tells the C++ compiler to compile the source file into a binary object `.o` file, but not link it yet.)
4. **Archived into a Library:** Once compiled into binary `.o` files, they are bundled together into a static library file (usually named `libhw_grpc_proto.a` on Linux/macOS).

What about the `.h` files listed inside `add_library`?

You might wonder why `${hw_proto_hdrs}` (`helloworld.pb.h`) and `${hw_grpc_hdrs}` (`helloworld.grpc.pb.h`) are listed in `add_library` if they aren't compiled directly.

The C++ compiler **never compiles header (`.h`) files directly**; it only compiles implementation (`.cc`) files. The headers are listed inside `add_library` for two main reasons:

1. **Dependency Tracking:** It tells CMake: "If these header files ever change, we need to recompile the `.cc` files that include them."
2. **IDE Support:** It tells IDEs (like VS Code or CLion) to display these generated headers in the project file explorer so you can easily click on them and read the generated code.

User prompt: Ok. Now I have one more doubt, or rather clarification. Now for our kv store project, a `cmakelists.txt` code was suggested right, but in the hello world example we were using `common.cmake` file as well (including it in the `cmakelists.txt`). Here for our kvstore project, is it not required? If it is not, then how are we replacing it? Here is the code in that `common.cmake` file: # Copyright 2018 gRPC authors. # Licensed under the Apache License, Version 2.0 (the "License"); # you may not use this file except in compliance with the License. # You may obtain a

```

copy of the License at # http://www.apache.org/licenses/LICENSE-2.0 # Unless required by applicable law or agreed to in writing, software #
distributed under the License is distributed on an "AS IS" BASIS, # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express
or implied. # See the License for the specific language governing permissions and # limitations under the License. # # cmake build file for C++
route_guide example. # Assumes protobuf and gRPC have been installed using cmake. # See cmake_externalproject/CMakeLists.txt for all-in-
one cmake build # that automatically builds all the dependencies before building route_guide. cmake_minimum_required(VERSION 3.16)
set(CMAKE_CXX_STANDARD 17) set(CMAKE_CXX_STANDARD_REQUIRED True) if(MSVC) add_definitions(-D_WIN32_WINNT=0x600)
endif() find_package(Threads REQUIRED) if(GRPC_AS_SUBMODULE) # One way to build a projects that uses gRPC is to just include the #
entire gRPC project tree via "add_subdirectory". # This approach is very simple to use, but there are some potential # disadvantages: # * it
includes gRPC's CMakeLists.txt directly into your build script # without and that can make gRPC's internal setting interfere with your # own
build. # * depending on what's installed on your system, the contents of submodules # in gRPC's third_party/ might need to be available (and
there might be # additional prerequisites required to build them). Consider using # the gRPC_*_PROVIDER options to fine-tune the expected
behavior. # # A more robust approach to add dependency on gRPC is using # cmake's ExternalProject_Add (see
cmake_externalproject/CMakeLists.txt). # Include the gRPC's cmake build (normally grpc source code would live # in a git submodule called
"third_party/grpc", but this example lives in # the same repository as gRPC sources, so we just look a few directories up) add_subdirectory(..../..
${CMAKE_CURRENT_BINARY_DIR}/grpc EXCLUDE_FROM_ALL) message(STATUS "Using gRPC via add_subdirectory.") # After using
add_subdirectory, we can now use the grpc targets directly from # this build. set(_PROTOBUF_LIBPROTOBUF libprotobuf) set(_REFLECTION
grpc++_reflection) set(_ORCA_SERVICE grpcpp_orca_service) if(CMAKE_CROSSCOMPILING) find_program(_PROTOBUF_PROTOC
protoc) else() set(_PROTOBUF_PROTOC ${CMAKE_CXX_COMPILER} protoc) endif() set(_GRPC_GRPCPP grpc++)
if(CMAKE_CROSSCOMPILING) find_program(_GRPC_CPP_PLUGIN_EXECUTABLE grpc_cpp_plugin) else()
set(_GRPC_CPP_PLUGIN_EXECUTABLE ${CMAKE_CXX_COMPILER} grpc_cpp_plugin) endif() elseif(GRPC_FETCHCONTENT) # Another way is to
use CMake's FetchContent module to clone gRPC at # configure time. This makes gRPC's source code available to your project, # similar to a
git submodule. message(STATUS "Using gRPC via add_subdirectory (FetchContent).") include(FetchContent) FetchContent_Declare( grpc
GIT_REPOSITORY https://github.com/grpc/grpc.git # when using gRPC, you will actually set this to an existing tag, such as # v1.25.0, v1.26.0
etc.. # For the purpose of testing, we override the tag used to the commit # that's currently under test. GIT_TAG
vGRPC_TAG_VERSION_OF_YOUR_CHOICE) FetchContent_MakeAvailable(grpc) # Since FetchContent uses add_subdirectory under the
hood, we can use # the grpc targets directly from this build. set(_PROTOBUF_LIBPROTOBUF libprotobuf) set(_REFLECTION
grpc++_reflection) set(_PROTOBUF_PROTOC ${CMAKE_CXX_COMPILER} protoc) set(_GRPC_GRPCPP grpc++) if(CMAKE_CROSSCOMPILING)
find_program(_GRPC_CPP_PLUGIN_EXECUTABLE grpc_cpp_plugin) else() set(_GRPC_CPP_PLUGIN_EXECUTABLE
${CMAKE_CXX_COMPILER} grpc_cpp_plugin) endif() else() # This branch assumes that gRPC and all its dependencies are already installed # on this
system, so they can be located by find_package(). # Find Protobuf installation # Looks for protobuf-config.cmake file installed by Protobuf's
cmake installation. option(protobuf_MODULE_COMPATIBLE TRUE) find_package(Protobuf CONFIG REQUIRED) message(STATUS "Using
protobuf ${Protobuf_VERSION}") set(_PROTOBUF_LIBPROTOBUF protobuf::libprotobuf) set(_REFLECTION grpc::grpc++_reflection)
if(CMAKE_CROSSCOMPILING) find_program(_PROTOBUF_PROTOC protoc) else() set(_PROTOBUF_PROTOC
${CMAKE_CXX_COMPILER} protoc) endif() # Find gRPC installation # Looks for gRPCConfig.cmake file installed by gRPC's cmake installation.
find_package(gRPC CONFIG REQUIRED) message(STATUS "Using gRPC ${gRPC_VERSION}") set(_GRPC_GRPCPP gRPC::grpc++)
if(CMAKE_CROSSCOMPILING) find_program(_GRPC_CPP_PLUGIN_EXECUTABLE grpc_cpp_plugin) else()
set(_GRPC_CPP_PLUGIN_EXECUTABLE ${CMAKE_CXX_COMPILER} grpc_cpp_plugin) endif() endif()

```

Response: No, the `common.cmake` file is **not required** for your custom `kv-store` project.

The reason the official gRPC "Hello World" example uses it is that it has to serve as a massive compatibility wrapper for thousands of different developers worldwide.

Why the Hello World Example Uses `common.cmake`

If you look closely at the `if-elseif-else` blocks in that file, you'll see that gRPC maintainers wrote three completely different strategies to find gRPC, depending on how a developer sets up their machine:

1. `GRPC_AS_SUBMODULE`: For people who cloned the entire gRPC source code directly into their project directory.
2. `GRPC_FETCHCONTENT`: For people who want CMake to automatically connect to the internet, clone gRPC from GitHub, and compile it on the fly.
3. `else()`: For people who have **already built and installed** gRPC and Protobuf on their local machine.

Because the example has to support all three scenarios, it uses `common.cmake` to figure out which environment is active, and then stores the result in proxy variables like `${_GRPC_GRPCPP}` and `${_PROTOBUF_LIBPROTOBUF}`.

How We Replace It for Your `kv-store` Project

For your custom key-value store, **we don't need all that complexity**. We already know exactly how gRPC is configured on your machine: you took the time to build and install it locally into your home environment directory (under the `../.local` prefix).

Therefore, we can throw away the submodule and internet-fetching branches entirely, and collapse the final `else()` block directly into your main project `CMakeLists.txt`. We can also use the modern CMake target names directly instead of assigning them to temporary helper variables.

Here is how the massive `common.cmake` wrapper file is replaced by just a few clean, explicit lines inside your root `kv-store/CMakeLists.txt`:

```

cmake_minimum_required(VERSION 3.16)
project(KeyValueStore CXX)

# 1. Replaces lines 23-24 of common.cmake (Enforces C++17)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 2. Replaces the entire "else()" block of common.cmake (Finds your local installation)
find_package(Protobuf CONFIG REQUIRED)
find_package(gRPC CONFIG REQUIRED)
find_package(Threads REQUIRED)

# 3. Code generation (Uses the direct target names instead of placeholders)
get_filename_component(kv_proto "proto/kv.proto" ABSOLUTE)

```

```

get_filename_component(kv_proto_path "${kv_proto}" PATH)

set(kv_proto_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.cc")
set(kv_proto_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.h")
set(kv_grpc_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.cc")
set(kv_grpc_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.h")

add_custom_command(
    OUTPUT "${kv_proto_srcs}" "${kv_proto_hdrs}" "${kv_grpc_srcs}" "${kv_grpc_hdrs}"
    COMMAND protocbuf::protoc # <--- Replaces ${_PROTOBUF_PROTOC}
    ARGS --grpc_out "${CMAKE_CURRENT_BINARY_DIR}"
        --cpp_out "${CMAKE_CURRENT_BINARY_DIR}"
        -I "${kv_proto_path}"
        --plugin=protoc-gen-grpc=${<TARGET_FILE:grpc::grpc_cpp_plugin> # <--- Replaces ${_GRPC_CPP_PLUGIN_E
        "${kv_proto}"
    DEPENDS "${kv_proto}")

include_directories("${CMAKE_CURRENT_BINARY_DIR}")

# 4. Create the proto library using direct modern targets
add_library(kv_grpc_proto ${kv_grpc_srcs} ${kv_proto_srcs})
target_link_libraries(kv_grpc_proto
    grpc::grpc++ # <--- Replaces ${_GRPC_GRPCPP}
    protobuf::libprotobuf # <--- Replaces ${_PROTOBUF_LIBPROTOBUF}
)

# 5. Build your actual Server Binary
add_executable(kv_server src/storage_engine.cpp src/server.cpp)
target_link_libraries(kv_server kv_grpc_proto Threads::Threads)

```

The Magic Parameter That Connects It

You might wonder: *If we stripped away all the searching logic, how does CMake know where to find gRPC on your computer?*

It happens when you trigger the build command in your terminal using the `CMAKE_PREFIX_PATH` flag:

```
cmake -DCMAKE_PREFIX_PATH=$HOME/.local ..
```

When you pass that flag, CMake automatically targets your isolated installation directory. The `find_package(gRPC CONFIG REQUIRED)` directive immediately scans `~/local` for `gRPCConfig.cmake`, hooks up the built-in compiler paths, and exposes `gRPC::grpc++` as a target.

By skipping `common.cmake`, your project's configuration becomes incredibly clean, explicit, and loads noticeably faster because CMake doesn't have to evaluate generic multi-environment check statements every time you run it!

User prompt: I think this code for `cmakelists.txt` is fine right?: """" `cmake_minimum_required(VERSION 3.8) project(KeyValueStore CXX) # Force the C++17 standard required by gRPC 1.70+ set(CMAKE_CXX_STANDARD 17) set(CMAKE_CXX_STANDARD_REQUIRED ON) # 1. Locate the gRPC and Protobuf packages installed in your local directory find_package(gRPC CONFIG REQUIRED) find_package(Protobuf CONFIG REQUIRED) # Paths where the compiler will generate the stub files on the fly set(proto_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.cc") set(proto_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.h") set(grpc_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.cc") set(grpc_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.h") # Find the exact code generator tools inside ~/local/bin find_program(PROTOC_program protoc REQUIRED) find_property(TARGET grpc::grpc_cpp_plugin PROPERTY LOCATION gRPC_CPP_PLUGIN_program) # 2. Automation: Tell CMake how to compile proto/kv.proto into C++ classes add_custom_command(OUTPUT "${proto_srcs}" "${proto_hdrs}" "${grpc_srcs}" "${grpc_hdrs}" COMMAND ${PROTOC_program} ARGS --cpp_out "${CMAKE_CURRENT_BINARY_DIR}" --grpc_out "${CMAKE_CURRENT_BINARY_DIR}" --plugin=protoc-gen-grpc="${gRPC_CPP_PLUGIN_program}" -I "${CMAKE_CURRENT_SOURCE_DIR}/proto" "${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.proto" DEPENDS "${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.proto") # Allow our files to find the auto-generated headers include_directories("${CMAKE_CURRENT_BINARY_DIR}" src) # 3. Define the core Storage Engine library add_library(storage_engine src/storage_engine.cpp src/storage_engine.h) # 4. Build the Server executable add_executable(kv_server src/server.cpp ${proto_srcs} ${grpc_srcs}) target_link_libraries(kv_server storage_engine gRPC::grpc++ Protobuf::libprotobuf) # 5. Build the Client executable add_executable(kv_client src/client.cpp ${proto_srcs} ${grpc_srcs}) target_link_libraries(kv_client gRPC::grpc++ Protobuf::libprotobuf) """" Now can you explain each and every line in this code: #pragma once #include <string> #include <unordered_map> #include <shared_mutex> #include <optional> class StorageEngine { public: void Put(const std::string& key, const std::string& value); std::optional<std::string> Get(const std::string& key) const; bool Delete(const std::string& key); private: mutable std::shared_mutex mutex_; std::unordered_map<std::string, std::string> data_;`

Response: Part 1: CMakeLists.txt Review & Corrections

Your `CMakeLists.txt` is close, but it contains **one fatal syntax error** and a couple of minor alignment issues with modern gRPC target naming conventions that will cause the build to fail.

The Issues:

1. **find_property does not exist:** There is no `find_property` command in CMake. To get the path of the gRPC plugin, you would normally have to use `get_target_property`. However, modern CMake provides a much cleaner tool called a **Generator Expression** (`${<TARGET_FILE:...>}`) which allows you to completely eliminate the search lines.
2. **Target Naming Case-Sensitivity:** Because you chose `find_package(Protobuf CONFIG REQUIRED)`, CMake loads the official configuration file provided by the Protobuf installation. The exported target name is **lowercase**: `protobuf::libprotobuf` (not capitalized `Protobuf::libprotobuf`).

3. **Double Compilation:** By including `${proto_srcs}` and `${grpc_srcs}` directly inside both `add_executable(kv_server ...)` and `add_executable(kv_client ...)`, you are instructing the C++ compiler to compile the proto code twice from scratch. It works, but it slows down build times.

The Corrected, Optimized `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.8)
project(KeyValueStore CXX)

# Force the C++17 standard required by gRPC 1.70+
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 1. Locate the gRPC and Protobuf packages installed in your local directory
find_package(gRPC CONFIG REQUIRED)
find_package(Protobuf CONFIG REQUIRED)
find_package(Threads REQUIRED) # Required for safe multi-threaded networking

# Paths where the compiler will generate the stub files on the fly
set(proto_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.cc")
set(proto_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.h")
set(grpc_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.cc")
set(grpc_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.h")

# 2. Automation: Tell CMake how to compile proto/kv.proto into C++ classes
add_custom_command(
    OUTPUT "${proto_srcs}" "${proto_hdrs}" "${grpc_srcs}" "${grpc_hdrs}"
    COMMAND protoc-gen-grpc=${TARGET_FILE:gRPC::grpc_cpp_plugin} # Clean modern target
    ARGS --cpp_out "${CMAKE_CURRENT_BINARY_DIR}"
         --grpc_out "${CMAKE_CURRENT_BINARY_DIR}"
         --plugin=protoc-gen-grpc=${TARGET_FILE:gRPC::grpc_cpp_plugin} # Safe Generator Expression
         -I "${CMAKE_CURRENT_SOURCE_DIR}/proto"
         "${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.proto"
    DEPENDS "${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.proto"
)

# Allow our files to find the auto-generated headers
include_directories("${CMAKE_CURRENT_BINARY_DIR}" src)

# 3. Compile the auto-generated Proto files into a single reusable library
add_library(kv_grpc_proto ${grpc_srcs} ${proto_srcs})
target_link_libraries(kv_grpc_proto gRPC::grpc++ protobuf::libprotobuf)

# 4. Define the core Storage Engine library
add_library(storage_engine src/storage_engine.cpp src/storage_engine.h)

# 5. Build the Server executable
add_executable(kv_server src/server.cpp)
target_link_libraries(kv_server storage_engine kv_grpc_proto Threads::Threads)

# 6. Build the Client executable
add_executable(kv_client src/client.cpp)
target_link_libraries(kv_client kv_grpc_proto Threads::Threads)
```

Part 2: Line-by-Line Explanation of `storage_engine.h`

Here is the line-by-line breakdown of your storage engine header, explaining the precise technical purpose of each instruction:

Line 1: `#pragma once`

- **What it is:** A preprocessor directive supported by almost all modern C++ compilers.
- **Why it's there:** It acts as a header guard. It tells the compiler to include this header file exactly **once** per compilation unit, even if multiple other files try to `#include` it. Without this, if `server.cpp` and `storage_engine.cpp` both include this file, the compiler would throw duplicate class definition errors.

Line 2: `#include <string>`

- **What it is:** Imports the Standard C++ String library (`std::string`).
- **Why it's there:** A key-value store relies fundamentally on text/binary keys and values. This header gives us access to the data type needed to store and pass keys and value data payloads across memory.

Line 3: `#include <unordered_map>`

- **What it is:** Imports the Standard associative container implemented as a Hash Table.
- **Why it's there:** This provides `std::unordered_map`, which serves as the physical storage memory block of our in-memory database. Hash tables provide highly optimized $O(1)$ average time complexity for insertions, lookups, and deletions, which ensures maximum performance.

Line 4: `#include <shared_mutex>`

- **What it is:** Imports the standard library synchronization primitive for reader-writer execution structures.
- **Why it's there:** Since a gRPC server spawns an independent thread pool to process network requests simultaneously, multiple client requests will hit the engine at the exact same moment. This header gives us access to `std::shared_mutex`, allowing multiple client threads to safely read from the engine at the same time while keeping write operations completely exclusive.

Line 5: `#include <optional>`

- **What it is:** Imports the C++17 value wrapper utility type (`std::optional`).
- **Why it's there:** When a client looks up a key using `Get()`, there are two possible outcomes: the key exists (return the string), or the key does not exist. Instead of returning a risky empty string or error codes, `std::optional<std::string>` elegantly represents a container that either cleanly holds a string or contains nothing (`std::nullopt`).

Line 7: `class StorageEngine {`

- **What it is:** Begins the structural definition blueprint of our `StorageEngine` object class wrapper.

Line 8: `public:`

- **What it is:** An access specifier keyword.
- **Why it's there:** Everything declared below this line is public, meaning external objects (like your gRPC network service handlers inside `server.cpp`) are granted direct permission to invoke these methods.

Line 9: `void Put(const std::string& key, const std::string& value);`

- **What it is:** Declares the database writing/insertion function interface.
- **Why it's written this way:** Passing arguments by reference (`const std::string&`) prevents the machine from copying the memory contents when passing them into the function.
- *Note on optimization:* If you want to use modern C++ move semantics to completely bypass data copies from the incoming network buffers, you can modify this signature to pass by value: `void Put(std::string key, std::string value);` and use `std::move()` inside the implementation.

Line 10: `std::optional<std::string> Get(const std::string& key) const;`

- **What it is:** Declares the read-only database lookup operation interface.
- **Why it's written this way:** Notice the trailing `const` keyword at the end. This is a strict contract promising the compiler that calling this method will **never modify** the state of the database. The `const` design allows multiple threads to safely invoke it simultaneously.

Line 11: `bool Delete(const std::string& key);`

- **What it is:** Declares the deletion interface method.
- **Why it's written this way:** It returns a boolean status flag—returning `true` if the item existed and was purged successfully, or `false` if the key was missing.

Line 13: `private:`

- **What it is:** An access specifier keyword.
- **Why it's there:** Enforces encapsulation. Everything below this line is hidden from the outside world. External files cannot touch or corrupt these variables directly; they must interact through the public functions above, which enforces thread safety.

Line 14: `mutable std::shared_mutex mutex_;`

- **What it is:** Instantiates the mutual exclusion read-write lock instance.
- **Why it's written this way:** The `mutable` keyword is critical here. In C++, a function marked `const` (like our `Get` method on Line 10) is forbidden from modifying any variable in the class. However, locking/unlocking a mutex technically mutates the inner state of that mutex. Marking the lock as `mutable` tells the compiler to grant special permission to modify the lock's state even inside read-only `const` methods.

Line 15: `std::unordered_map<std::string, std::string> data_;`

- **What it is:** Instantiates the actual database storage memory structure.
- **Why it's there:** This maps your unique identifier `std::string` keys directly to their respective `std::string` binary value blobs in the host machine's RAM space.

Line 16: `};`

- **What it is:** Closes out the class blueprint definition declaration block.

User prompt: ok. In the `CMakeLists.txt` code you have given, why did we add threads, and what about all these link libraries that were present in the hello world example: `absl::check absl::flags absl::flags_parse absl::log absl::log_initialize ${_REFLECTION}`

```
$_GRPC_GRPCPP} $_PROTOBUF_LIBPROTOBUF}
```

Response: 1. Why did we add **Threads**?

To put it simply: **gRPC is inherently multi-threaded, and C++ needs to know how to link the operating system's threading libraries.**

```
find_package(Threads REQUIRED)
...
target_link_libraries(kv_server ... Threads::Threads)
```

How gRPC uses threads:

When you start a gRPC server (e.g., when you call `server->Wait()`), gRPC doesn't run your network loop on a single CPU thread. Instead, it spawns a background **thread pool** to handle hundreds or thousands of incoming client requests simultaneously.

Why CMake needs this:

On operating systems like Linux, multi-threading is handled by the POSIX Threads library (`pthread`). If you compile a C++ program that utilizes system threads without telling the compiler/linker to link against `pthread`, the build will crash with massive linker errors:

```
undefined reference to 'pthread_create'
undefined reference to 'pthread_join'
```

Using `find_package(Threads REQUIRED)` is the modern, cross-platform way to tell CMake: "Find the threading library of whatever operating system we are compiling on (Windows threads, POSIX threads on Linux, etc.) and safely link it to our executables via `Threads::Threads`."

2. What happened to all those other link libraries?

The Hello World example links against a long list of libraries. Let's break down exactly what happened to each one in our simplified `CMakeLists.txt`:

A. Google Abseil Libraries (`absl::check`, `absl::flags`, `absl::flags_parse`, `absl::log`, `absl::log_initialize`)

- **What they are:** [Abseil](#) is an open-source collection of C++ library code designed by Google to augment the C++ standard library.
- **Why Hello World used them:** In Google's official files (`greeter_client.cc` and `greeter_server.cc`), they chose to use Abseil to parse command-line arguments (e.g., passing a custom port like `--port=50051`) and to handle system logging.
- **Why we removed them:** These libraries are **completely optional** for your custom key-value store. If your `server.cpp` and `client.cpp` use standard C++ features for command-line arguments (like reading `argv[1]`) and standard console outputs (like `std::cout` or `std::cerr`), **you do not need to link Abseil**. Stripping them out keeps your compilation times fast and prevents your build from depending on excessive external libraries.

B. The Reflection Library (`$_REFLECTION`)

- **What it is:** gRPC Server Reflection is an optional service. When enabled on a server, it allows external command-line tools (like `grpcurl` or API clients like Postman) to query your server dynamically and ask: "Hey, what RPC methods do you support?" without needing to have the `.proto` file pre-installed.
- **Why we removed it:** While incredibly helpful for debugging, reflection is not strictly necessary to make your key-value store work. To keep your initial `CMakeLists.txt` as simple as possible, we left it out. (Note: If you want to use tools like `grpcurl` to test your database later, we can easily add `gRPC::grpc++_reflection` back with just one line of code!)

C. The Core gRPC & Protobuf Targets (`$_GRPC_GRPCPP` and `$_PROTOBUF_LIBPROTOBUF`)

- **What they are:** These are the actual, fundamental libraries that make gRPC and Protobuf work.
- **They did not disappear!** We simply stopped using their complex "placeholder" variable names and replaced them with their direct, modern CMake target names:

Variable in <code>common.cmake</code>	Direct Target in our <code>CMakeLists.txt</code>	What it actually links
<code>\$_GRPC_GRPCPP</code>	<code>gRPC::grpc++</code>	The core C++ gRPC runtime.
<code>\$_PROTOBUF_LIBPROTOBUF</code>	<code>protobuf::libprotobuf</code>	The core Protocol Buffers serialization library.

By directly linking `gRPC::grpc++` and `protobuf::libprotobuf` inside your `CMakeLists.txt`, we achieved the exact same result as the Hello World example but bypassed the confusing layers of redirection from `common.cmake`.

User prompt: Threads was not present in the hello world example right. So it might be not problem to not include in this kv store `cmakelist.txt` file too right??

Response: Actually, it **was** present in the Hello World example! It was just hiding inside that `common.cmake` file.

If you look back at line 29 of the `common.cmake` code you shared earlier, you'll spot this exact line:

```
find_package(Threads REQUIRED)
```

Because the Hello World project automatically included `common.cmake` at the top, it quietly inherited the system threads dependency without making you write it out in the main file. Now that we are building a clean, independent `CMakeLists.txt` for your key-value store without using `common.cmake`, we have to explicitly bring that line over.

What happens if you try to remove it?

If you delete `Threads` from your project's `CMakeLists.txt`, your code will actually pass the initial compilation stage cleanly, but it will **crash violently at the very end during the linking stage**.

You will get a wall of confusing terminal text filled with errors like this:

```
/usr/bin/ld: undefined reference to 'pthread_create'
/usr/bin/ld: undefined reference to 'pthread_join'
collect2: error: ld returned 1 exit status
```

Why gRPC strictly demands it

Even if your own custom `server.cpp` code looks completely sequential and doesn't seem to use threads, the **gRPC runtime library** (`gRPC::grpc++`) **absolutely does**.

When a client hits your key-value store to run a `Get` or `Put` request, gRPC's core engine catches that network packet, pulls a background thread out of its internal CPU thread pool, and assigns that thread to execute your storage engine code. Because gRPC relies entirely on the host operating system's low-level threading primitives (like `libpthread` on Linux) to pull this off, the C++ compiler refuses to build the final server binary unless you explicitly link `Threads::Threads`.

Keeping it in your `CMakeLists.txt` ensures your project remains robust, multi-threaded, and completely cross-platform!

User prompt: I see `find_package(Threads)` is present in the `common.cmake`, but I don't see that it was anywhere linked to the client and server .cc files in the helloworld example right?

Response: Wow, incredible catch! You have an excellent eye for detail. You are 100% correct—if you look at the `CMakeLists.txt` for the official `helloworld` example, `Threads::Threads` is never explicitly listed in the `target_link_libraries` section for the client or server.

So, how on earth did it compile for them without throwing an error?

The answer comes down to a fundamental CMake feature called **Transitive Dependencies** (or dependency chaining).

The Secret: Transitive Dependencies

In modern CMake, when a library creator packages their software, they don't just export the compiled binary; they also export a list of everything their library needs to survive. This is done using the `INTERFACE` or `PUBLIC` keywords.

Because gRPC is fundamentally multi-threaded, the maintainers wrote gRPC's internal configuration exports to say: *"Whoever links against `gRPC::grpc++` must also automatically pull in the system's threading library."*

When the `helloworld` example executes this line:

```
target_link_libraries(greeter_server gRPC::grpc++)
```

CMake looks under the hood at `gRPC::grpc++`, notices its hidden internal checklist, and quietly injects the operational system's multi-threading flags (like `-pthread` on Linux) into the compilation command behind the scenes.

If gRPC pulls it in automatically, why did we explicitly add it?

While you *can* technically get away with leaving it out and letting gRPC pass it down like a hand-me-down, explicitly adding `find_package(Threads)` and linking it to your targets is standard best practice for two critical reasons:

1. Your own code uses threads directly

There is a golden rule in software engineering build systems: **If your code explicitly uses a feature, your build file should explicitly declare it**. Inside your `storage_engine.h`, you wrote this line:

```
#include <shared_mutex>
mutable std::shared_mutex mutex_;
```

`std::shared_mutex` is a standard C++ synchronization primitive that compiles down to the operating system's thread lock mechanisms. Because **your** code explicitly requests thread locks independent of anything gRPC is doing, your `CMakeLists.txt` should confidently own that dependency by declaring `Threads::Threads`.

2. Protection against fragile updates

Relying on another library to hand you your dependencies is risky. If gRPC maintainers modify their exported configuration files in a future version to link threads privately (`PRIVATE`) rather than publicly (`INTERFACE`), the transitive chain breaks instantly. Your key-value store would suddenly fail to compile out of nowhere after an update.

Explicitly linking `Threads::Threads` acts as an insurance policy. It makes it clear to anyone reading your architecture layout that this project requires multi-threaded capabilities to function, regardless of how its underlying dependencies change!

User prompt: so is `find_package(Threads)` enough, or should we even link it to the server and client? Or is `find_package(Threads)` is also not

required? Also does `find_package(Threads)` just find the location of `Threads` or does it also automatically make it available to files like `storage_engine.h`? Also, if I run (create .o file and then execute it) just the `storage_engine.cc` as a separate cpp file just in vs code using the gcc compiler, will it automatically use threads, or should I explicitly do something inorder for its threads code to work?

Response: 1. Does `find_package(Threads)` automatically make threads available?

No, it does **not** automatically make threads available to your code or files like `storage_engine.h`.

`find_package(Threads)` is purely a **locator**. Think of it like running a search on your computer to see if a file exists. When you run `find_package(Threads REQUIRED)`, CMake searches the host operating system to find where the system threading library (like `libpthread` on Linux) is located. Once found, CMake creates a hidden configuration object named `Threads::Threads` that stores the exact paths and compiler flags needed to use threads.

However, your C++ targets (`kv_server` and `kv_client`) have no idea this object exists until you explicitly hand it to them. You **must** link it.

If you use `find_package(Threads)` but delete `Threads::Threads` from `target_link_libraries`, the `find_package` line becomes completely useless dead code.

2. Is `find_package(Threads)` even required for this project?

Technically, **no**. You could delete both `find_package(Threads REQUIRED)` and `Threads::Threads` from your `CMakeLists.txt` right now, and the project would still compile successfully.

This works *only* because you are linking against `gRPC::grpc++`. As we discovered, the gRPC maintainers already included `Threads` inside their own exported configuration. Because CMake passes down dependencies transitively, your server and client borrow the threading flags directly from gRPC.

However, explicitly keeping both lines is highly recommended for professional code. Your `storage_engine.h` uses `std::shared_mutex` entirely independently of gRPC. If you ever decide to copy your `StorageEngine` code into a different project that doesn't use gRPC, your build will immediately break if you haven't explicitly declared `Threads` in your build script.

3. Compiling `storage_engine.cc` directly in VS Code via GCC

If you bypass CMake entirely and try to compile `storage_engine.cc` (along with a basic `main.cpp` to test it) directly using `g++` in a Linux/macOS terminal or a VS Code task, **it will not work automatically**. If you run a standard compilation command like this:

```
g++ -std=c++17 storage_engine.cpp main.cpp -o main
```

The GCC compiler will throw a severe linker error at the very end:

```
undefined reference to 'pthread_rwlock_init'
undefined reference to 'pthread_rwlock_wrlock'
collect2: error: ld returned 1 exit status
```

What you must do explicitly:

To make it work, you must append the `-pthread` flag to your `g++` command. This flag tells GCC to do two things:

1. Define the internal preprocessor macros (like `REENTRANT`) required for the C++ Standard Library's threading headers (`<shared_mutex>`, `<mutex>`, `<thread>`) to initialize safely.
2. Tell the system linker to bind the actual `libpthread.so` system library to your final executable.

The correct command to run your storage engine standalone is:

```
g++ -std=c++17 -pthread storage_engine.cpp main.cpp -o main
```

Setting it up in VS Code:

If you are using the VS Code "Run C/C++ File" button (which uses a `tasks.json` file under the hood), you need to open your `.vscode/tasks.json` file and make sure `-pthread` is explicitly included in the `args` array:

```
{
  "tasks": [
    {
      "type": "cppbuild",
      "label": "C/C++: g++ build active file",
      "command": "/usr/bin/g++",
      "args": [
        "-tasks",
        "-std=c++17",
        "-pthread", // <-- YOU MUST EXPLICITLY ADD THIS LINE
        "${file}",
        "-o",
        "${fileDirname}/${fileBasenameNoExtension}"
      ],
      ...
    }
  ]
}
```

User prompt: is there no `#include` line of code that can include the code for threads directly? Also in `common.cmake` file, they have done `find_package(Threads)` but have they used it anywhere??

Response: These are two of the most profound questions every C++ developer encounters when they start moving from simple single-file programs to complex, real-world systems. Let's demystify both.

Part 1: Why isn't an `#include` line enough for threads?

It seems completely logical: if you can `#include <thread>` or `#include <shared_mutex>`, why doesn't the compiler just copy-paste the thread code right there and make it work?

The reason comes down to how C++ splits its work into two entirely separate phases: **Compilation** and **Linking**.

1. The `#include` phase (Compilation) is just a "Blueprint"

When you write `#include <shared_mutex>`, you are only bringing in a header file. Header files do **not** contain the actual working machine code to create a thread. They only contain declarations—a list of rules that tell the compiler:

- "There is a class called `std::shared_mutex`."
- "It has a method called `.lock()` and `.unlock()`."

This allows the compiler to check your syntax and verify that you are using the commands correctly. The compiler says, "Okay, this looks legally correct," and generates a temporary object file (`.o`).

2. The Linking phase is the "Actual Engine"

Creating, managing, and destroying CPU threads requires deep interaction with the Operating System kernel. The actual machine code that handles this doesn't live in a text header file—it resides inside the operating system's pre-compiled system libraries (like `libpthread.so` on Linux).

The **Linker** is the tool responsible for stitching your temporary object files together with the operating system's libraries.

- Without the `-pthread` flag or `Threads::Threads`, the Linker looks at your code, sees a request to initialize a mutex lock, and says: "The compiler told me this was a valid command, but I have no idea where the actual OS code is to execute this lock." **Result: Linker Error.**
- By adding the threading flag or library, you are telling the linker exactly where to find the physical operating system code to execute those commands.

Part 2: Did `common.cmake` use `Threads` anywhere?

You caught them red-handed! If you scour the official gRPC `common.cmake` file, you will find that `find_package(Threads REQUIRED)` is declared at the top, but `Threads::Threads` is **never explicitly linked to any target** inside that file or the main `CMakeLists.txt`.

Why is it there? There are two main reasons why engineers write code like this:

1. It acts as an early "Sanity Check" (Fail-Fast Principle)

Compiling a massive framework like gRPC can take a long time. If a developer attempts to compile this project on a highly stripped-down, specialized embedded system or an ancient operating system that completely lacks multi-threading libraries, the build will eventually crash.

By placing `find_package(Threads REQUIRED)` at the very top of `common.cmake`, CMake will scan the system immediately. If it can't find thread support, **it will crash within 1 second** and tell the developer why, rather than wasting 10 minutes compiling files only to fail at the very end.

2. It is a "Legacy Leftover" (Technical Debt)

Large open-source projects like gRPC have been maintained for a decade. In older versions of CMake and gRPC, developers *did* have to manually link `Threads::Threads` to every single client and server executable.

As CMake evolved, gRPC upgraded its build architecture to use **transitive dependencies** (meaning the core `gRPC::grpc++` target started carrying the thread dependency implicitly, as we discovered earlier). When the gRPC maintainers modernized their code, they removed the manual linking lines from the executables, but they forgot to clean up the `find_package(Threads REQUIRED)` line from the shared template file.

Summary for your project:

- You **must** keep using `#include <shared_mutex>` so the compiler understands your syntax.
- You **must** keep the threading configuration (`-pthread` in GCC or `Threads::Threads` in CMake) so the linker can bridge your code to the host operating system.
- You can confidently ignore the way `common.cmake` handled it—your clean, explicit setup is far more accurate and professional!

User prompt: ok. And hey, look, I am new to programming and especially totally new to grpc, protobuf, and non-basic cpp (like threads, mutex, data structures, etc.). So please explain totally and clearly with full detail. Also make sure you don't hallucinate. Also be updated with latest information. now, explain me this code: `#pragma once #include <string> #include <unordered_map> #include <shared_mutex> #include <optional> class StorageEngine { public: void Put(const std::string& key, const std::string& value); std::optional<std::string> Get(const std::string& key) const; bool Delete(const std::string& key); private: mutable std::shared_mutex mutex_; std::unordered_map<std::string,`

```
std::string> data_;};
```

Response: Welcome to C++ systems programming! This header file (`.h`) represents the absolute heart of your key-value store: the **Storage Engine**.

Even though it is only 15 lines of code, it uses intermediate and advanced C++ features designed for high performance, memory efficiency, and multi-threaded safety. When your gRPC server receives network requests over the internet from clients, it will hand those requests directly to this class to process.

Let's break this down concept-by-concept, line-by-line, assuming no prior background.

Phase 1: The Header Configuration (`#pragma once` & `#include`)

```
#pragma once
```

- **What it does:** This is called a **preprocessor directive**. It acts as a safety guard.
- **Why it's needed:** In large C++ projects, multiple code files might try to look at this header file simultaneously. If two files import this header into the main compilation file, the compiler will panic and complain about "duplicate definitions" of the `StorageEngine` class. `#pragma once` tells the compiler: "Only read this file exactly once during compilation, no matter how many times it gets imported."

```
#include <string>
#include <unordered_map>
#include <shared_mutex>
#include <optional>
```

These lines import tools from the **C++ Standard Library (std)**. Think of them as importing specialized modules:

1. `<string>`: Gives us `std::string`, which allows us to safely handle text (keys and values) without dealing with low-level character arrays.
2. `<unordered_map>`: This is our database structure. In computer science, this is a **Hash Table**. It stores data as *Key* → *Value* pairs. It is called "unordered" because the computer doesn't store them alphabetically; it uses a mathematical calculation (a hash function) to instantly jump straight to where a key is kept in memory. This gives it a speed called $O(1)$ **time complexity**, meaning it takes the same fraction of a microsecond to find a key whether your database has 10 items or 10 million items.
3. `<shared_mutex>`: This is the tool that makes our code safe for **multi-threading** (running multiple tasks on multiple CPU cores at the exact same time). We will deep dive into this below.
4. `<optional>`: A modern safety feature added in C++17. It represents a value that *might or might not exist*.

Phase 2: The Public Interface (`public:`)

In C++, a `class` is a blueprint. The `public:` keyword means: "Any code outside this class (like our gRPC server network code) is allowed to call these functions."

```
void Put(const std::string& key, const std::string& value);
```

- **Purpose:** Adds a new key-value pair to the database, or overwrites an existing key.
- **Advanced C++ Concept (`const std::string&`):** Notice it doesn't say `std::string key`. It has a `const` and an `&` symbol. This is an essential optimization called **Pass-by-Reference-to-Const**.
 - If you pass a variable normally, C++ makes a complete physical **copy** of it in memory. If a client sends a 5-megabyte value string, passing by value duplicates it, wasting CPU time and memory.
 - The `&` symbol means **Reference**. Instead of copying the string, it passes a tiny memory pointer (an address) directly to where the original string is sitting.
 - Because passing a reference gives the function direct access to the original data, the function *could* accidentally modify it. To prevent this, we add `const`. This guarantees to the compiler that the function will treat the input as read-only.

```
std::optional<std::string> Get(const std::string& key) const;
```

- **Purpose:** Looks up a key and returns its value.
- **Advanced C++ Concept 1 (`std::optional`):** What happens if a client asks for a key that does *not* exist in our database? In older C++ versions, you might return an empty string `" "` or a null pointer. But `" "` could be a valid value that someone actually stored! `std::optional<std::string>` solves this perfectly. It is a wrapper box. If the key exists, it returns a box containing the string. If the key doesn't exist, it returns an empty box (`std::nullopt`). The gRPC server can check if the box is empty and safely send a "Key Not Found" error back over the network.
- **Advanced C++ Concept 2 (The trailing `const`):** Notice the `const` sitting at the very end of the line, right before the semicolon. This tells C++ that the `Get` function is a **read-only function**. It promises that calling `Get` will not change any internal data within the database.

```
bool Delete(const std::string& key);
```

- **Purpose:** Removes a key from the map. It returns `true` if the key existed and was successfully deleted, or `false` if the key didn't exist in the first place.

Phase 3: The Private Engine Rooms (`private:`)

The `private:` keyword implements a core principle of software engineering called **Encapsulation**. It means: "No outside file or function is allowed to touch or see these variables directly." Outside code must interact through our public functions (`Put`, `Get`, `Delete`).

```
std::unordered_map<std::string, std::string> data_;
```

- This is the actual container where your data physically lives in the computer's RAM. It maps a string key to a string value.
- *Style Note:* The trailing underscore (`data_`) is a standard industry convention indicating that this variable is a private class member.

```
mutable std::shared_mutex mutex_;
```

This is the most advanced line in the file. It implements **Thread Safety**.

1. Why do we need a Mutex?

gRPC is naturally multi-threaded. If client A calls `Put("username", "alice")` at the exact same physical microsecond that client B calls `Delete("username")`, both threads will rush into the `std::unordered_map` data structure simultaneously. This causes a catastrophic event called a **Data Race**. The internal memory pointers of the map will become corrupted, causing the entire server program to instantly crash. A Mutex (Mutual Exclusion) acts like a lock on a door.

2. Why a Shared Mutex? (The Reader-Writer Lock pattern)

A standard mutex locks the door completely: only one thread can be inside at a time. But in a database, if 100 clients want to run `Get` (Read) at the same time, they aren't changing the data, so it is completely safe to let them all read simultaneously!

`std::shared_mutex` creates two distinct modes:

- **Exclusive Lock (for Writes/Deletes):** When `Put` or `Delete` is running, the thread requests an exclusive lock. Everyone else (readers and writers) is forced to halt and wait outside until it finishes modifying the map.
- **Shared Lock (for Reads):** When `Get` is running, it requests a shared lock. 100 different threads running `Get` can hold the lock simultaneously. However, if a `Put` request arrives, it waits until all readers exit before executing.

3. Why the mutable keyword?

This solves a structural paradox in C++. Earlier, we marked the `Get` function as a trailing `const` because it's a read-only method. C++ strictly enforces this: inside a `const` function, you are completely forbidden from changing the state of *any* variable inside the class.

However, when `Get` executes, it must lock and unlock our `mutex_`. Locking a mutex changes its internal state from "unlocked" to "locked". The compiler catches this and throws a compilation error: *"You promised Get was read-only, but you are changing the state of your mutex!"*

By tagging the variable with `mutable`, you tell the C++ compiler: *"This specific variable is exempt from standard const rules. Let me modify it even inside read-only functions."* It is almost exclusively used for thread locks in professional systems engineering.

User prompt: now explain this code: `#include "storage_engine.h" void StorageEngine::Put(const std::string& key, const std::string& value) { std::unique_lock lock(mutex_); // exclusive lock for writes data_[key] = value; } std::optional<std::string> StorageEngine::Get(const std::string& key) const { std::shared_lock lock(mutex_); // shared lock — multiple readers OK auto it = data_.find(key); if (it == data_.end()) return std::nullopt; return it->second; } bool StorageEngine::Delete(const std::string& key) { std::unique_lock lock(mutex_); return data_.erase(key) > 0; }`

Response: This `.cpp` file is the actual implementation file. While the header file (`.h`) was the blueprint that *declared* what functions exist, this file contains the actual muscle—the exact code that executes when someone calls `Put`, `Get`, or `Delete`.

Before we look at the functions individually, we need to understand the single most important C++ pattern used in all three functions: **RAII Scope-Based Locking**.

The Magic Concept: RAII and Auto-Unlocking

In older programming languages (or old C code), if you wanted to protect a resource with a mutex, you had to write code like this:

```
mutex_.lock();
// ... do some work ...
mutex_.unlock(); // <-- Danger zone!
```

This is incredibly dangerous. If your code between `lock()` and `unlock()` has a bug, throws an error, or exits early via a `return` statement, the `mutex_.unlock()` line will never be reached. The door stays locked forever, and your entire application freezes up (a bug called a **Deadlock**).

Modern C++ solves this using a design pattern called **RAII** (Resource Acquisition Is Initialization).

Instead of locking manually, we create a temporary local variable (a "lock guard" object) inside the function.

1. The moment that variable is born, it automatically locks the mutex.
2. The moment the function reaches its closing curly brace `}`, that local variable is destroyed, and its cleanup code automatically unlocks the mutex. **It is physically impossible to forget to unlock.**

Let's look at how this is applied across the three functions.

1. Connecting to the Blueprint

```
#include "storage_engine.h"
```

- **What it means:** Notice the quotation marks `" "` instead of angle brackets `< >`. Angle brackets tell the compiler to look in the system libraries (like standard C++ tools). Quotation marks tell the compiler: *"Look inside our own custom project folder for this file."* This links our

logic file to the definitions class we wrote earlier.

2. The Put Function (Writing Data)

```
void StorageEngine::Put(const std::string& key, const std::string& value) {
    std::unique_lock lock(mutex_); // exclusive lock for writes
    data_[key] = value;
}
```

- **StorageEngine::Put**: The double colon `::` is called the **Scope Resolution Operator**. It tells the compiler: *"This isn't just a random global function named Put. This is the official body of the Put function belonging inside the StorageEngine class."*
- **std::unique_lock lock(mutex_)**: This creates an **Exclusive Lock**. Because a write operation changes the underlying memory map structure, absolutely no other thread is allowed to be reading or writing at this exact microsecond.
 - *Modern C++17 Note*: In older C++, you had to write `std::unique_lock<std::shared_mutex> lock(mutex_)`. Thanks to a modern feature called **CTAD** (Class Template Argument Deduction), the compiler automatically looks at `mutex_`, figures out it is a `std::shared_mutex`, and configures the template for you.
- **data_[key] = value**: This accesses our private hash map (`data_`). If the `key` does not exist, the map creates a new entry and assigns it the `value`. If the `key` already exists, it cleanly overwrites the old value with the new one.
- **The Closing Brace**: The function ends. The temporary variable `lock` immediately dissolves out of memory, instantly releasing the exclusive lock so other threads waiting in line can jump in.

3. The Get Function (Reading Data)

```
std::optional<std::string> StorageEngine::Get(const std::string& key) const {
    std::shared_lock lock(mutex_); // shared lock - multiple readers OK
    auto it = data_.find(key);
    if (it == data_.end()) return std::nullopt;
    return it->second;
}
```

- **std::shared_lock lock(mutex_)**: Unlike `unique_lock`, this creates a **Shared Lock**. If 50 different clients make a gRPC request to read data at the same time, 50 different threads will execute this function simultaneously. Because it's a shared lock, the operating system allows all 50 threads to walk right through the door together. However, if a `Put` request arrives, the operating system halts the `Put` thread until these readers finish up.
- **auto it = data_.find(key)**: We do *not* use `data_[key]` to read data. In C++, if you use brackets `[]` on a map and the key isn't there, the map will accidentally *create* an empty blank entry for that key! To keep this function truly read-only (`const`), we use the `.find()` method. This returns an **Iterator** (which you can think of as a safe pointer or a target bookmark pointing to where the item lives inside the map).
- **if (it == data_.end()) return std::nullopt**: If the map searches all its items and can't find your key, it returns a special placeholder called `data_.end()`. This placeholder represents the imaginary "finish line" boundary just past the final element of the map. If our iterator `it` hits the finish line, it means the key does not exist. We return `std::nullopt` (our empty wrapper box).
- **return it->second**: In C++, internal map elements are stored as a structure called a `std::pair`. A pair has two variables inside it: `.first` (which holds the Key) and `.second` (which holds the Value). Since we want to return the actual value string to our client, we return `it->second`. C++ automatically takes this raw string and packages it into the `std::optional` box before sending it out.

4. The Delete Function (Removing Data)

```
bool StorageEngine::Delete(const std::string& key) {
    std::unique_lock lock(mutex_);
    return data_.erase(key) > 0;
}
```

- **std::unique_lock lock(mutex_)**: Because deleting an item changes the map structure (shrinking it and rearranging internal memory buckets), we must use an **Exclusive Lock** again to keep it safe from concurrent readers.
- **data_.erase(key)**: The `.erase()` method tells the map to find the key and physically wipe it out of memory.
- **return data_.erase(key) > 0**: How do we return a `true` or `false` value here? In a standard C++ `std::unordered_map`, keys are unique—you can only ever have zero or one instance of a specific key name. The `.erase()` function returns an integer counting how many items it deleted.
 - If the key existed and was successfully wiped out, `.erase()` returns 1. Because `1 > 0` is mathematically **true**, the function returns `true`.
 - If the key never existed in the database, `.erase()` does nothing and returns 0. Because `0 > 0` is mathematically **false**, the function returns `false`.

User prompt: ok, cool. Right now we are in phase 1 of our project, and in step 3 (Implement the storage engine (in isolation, no networking yet)). This step 3 also says: "Write a standalone unit test for this BEFORE wiring up gRPC. Use plain assert() or a simple test framework (Catch2 or GoogleTest — gRPC already pulls in GoogleTest as a dependency, so it's free to use). Test: Put then Get returns the value Get on missing key returns nullopt Delete removes it, second delete returns false (Optional but valuable) spin up 10 threads hammering Put/Get on overlapping keys, run under ThreadSanitizer (-fsanitize=thread) to catch races This step matters because once you wire in networking, debugging concurrency bugs through gRPC becomes much harder. Isolate and verify the core logic first." So let's do it. Tell me exactly and clearly in detail how to do it.

Response: To verify your storage engine in isolation before adding network layers, you will write a clean, zero-dependency unit test file using modern C++ features.

This guide sets up functional assertions, builds a stress test with **10 concurrent threads** targeting overlapping keys, and configures **ThreadSanitizer (TSan)** via CMake to prove your code is free of data races.

Step 1: Establish Your Project Directory Structure

Make sure your folder matches a clean layout like this:

```
kv-store/
├── CMakeLists.txt
├── src/
│   ├── storage_engine.h
│   └── storage_engine.cpp
└── tests/
    └── test_storage.cpp
```

Step 2: Write the Test Runner (tests/test_storage.cpp)

Create a new file inside your `tests/` directory. This script uses standard `<cassert>` for immediate feedback and utilizes standard C++ lambda threads to stress the memory map.

```
#include "../src/storage_engine.h"
#include <cassert>
#include <iostream>
#include <thread>
#include <vector>
#include <string>

// Test Case 1: Functional core verification
void test_functional() {
    StorageEngine engine;

    // 1. Get on missing key returns nullopt
    assert(engine.Get("missing_key") == std::nullopt);

    // 2. Put then Get returns the correct value
    engine.Put("cluster_size", "6_nodes");
    auto val = engine.Get("cluster_size");
    assert(val.has_value());
    assert(val.value() == "6_nodes");

    // 3. Delete removes the key
    assert(engine.Delete("cluster_size") == true);
    assert(engine.Get("cluster_size") == std::nullopt);

    // 4. A second delete on a wiped key must return false
    assert(engine.Delete("cluster_size") == false);

    std::cout << "[✓] Functional validation passed successfully." << std::endl;
}

// Test Case 2: Concurrency stress-testing
void test_concurrency() {
    StorageEngine engine;
    const int num_threads = 10;
    const int iterations = 5000;
    std::vector<std::thread> threads;

    std::cout << "[!] Launching " << num_threads
              << " threads to hammer overlapping keys..." << std::endl;

    for (int i = 0; i < num_threads; ++i) {
        threads.emplace_back([&engine, i]() {
            for (int j = 0; j < iterations; ++j) {
                // By forcing modulo 3, all 10 threads fiercely compete
                // for the exact same 3 key slots simultaneously.
                std::string key = "shared_key_" + std::to_string(j % 3);
                std::string val = "thread_" + std::to_string(i) + "_iter_" + std::to_string(j);

                // Mix writes, reads, and deletes randomly/sequentially
                engine.Put(key, val);

                auto res = engine.Get(key);

                if (j % 7 == 0) {
                    engine.Delete(key);
                }
            }
        });
    }
}
```

```

    // Synchronize and wait for all threads to cross the finish line
    for (auto& t : threads) {
        t.join();
    }

    std::cout << "[✓] Concurrency stress test completed without crashing." << std::endl;
}

int main() {
    std::cout << "=====" << std::endl;
    std::cout << "    RUNNING STORAGE ENGINE ISOLATION TESTS    " << std::endl;
    std::cout << "=====" << std::endl;

    test_functional();
    test_concurrency();

    std::cout << "=====" << std::endl;
    std::cout << " ALL CORE VERIFICATIONS PASSED SUCCESSFULLY! " << std::endl;
    std::cout << "=====" << std::endl;
    return 0;
}

```

Step 3: Wire the Test Executable Into CMakeLists.txt

To compile this test target cleanly alongside any current code, append the following target declaration to the bottom of your root `CMakeLists.txt` file.

Notice how we explicitly attach `-fsanitize=thread` to both the compiler flags and the linker flags. We also use `-g` (to keep debug symbols) and `-O1` (recommended by ThreadSanitizer to keep stack traces fast yet accurate).

```

# Look for standard operating system thread libraries (pthread)
find_package(Threads REQUIRED)

# Declare the standalone test binary target
add_executable(test_storage
    src/storage_engine.cpp
    tests/test_storage.cpp
)

# Link standard system threads to the binary
target_link_libraries(test_storage PRIVATE Threads::Threads)

# Enforce compiler instrumentation for ThreadSanitizer
target_compile_options(test_storage PRIVATE
    -fsanitize=thread
    -g
    -O1
)

# Enforce linker instrumentation for ThreadSanitizer
target_link_options(test_storage PRIVATE
    -fsanitize=thread
)

```

Step 4: Build and Execute

Run the following build sequence from your terminal to compile and evaluate the test runner.

```

# 1. Enter the root project folder
cd kv-store

# 2. Create a clean out-of-source build workspace
mkdir -p build && cd build

# 3. Configure the build environment
cmake ..

# 4. Compile only the test suite target
make test_storage

# 5. Execute the test
./test_storage

```

Step 5: How to Read the Results

Scenario A: Perfect Success (What you expect to see)

If your `std::shared_mutex` implementation works flawlessly, ThreadSanitizer will remain quiet throughout execution, allowing your print statements to display naturally:

```

=====
    RUNNING STORAGE ENGINE ISOLATION TESTS
=====

```

```
[✓] Functional validation passed successfully.
[!] Launching 10 threads to hammer overlapping keys...
[✓] Concurrency stress test completed without crashing.
=====
ALL CORE VERIFICATIONS PASSED SUCCESSFULLY!
=====
```

Scenario B: A Data Race Exists (If protection fails)

If you remove the mutex locks or misconfigure the read/write permissions, ThreadSanitizer will intercept the unsafe memory access at runtime, stop execution, and dump a detailed report directly into your terminal:

```
=====
WARNING: ThreadSanitizer: data race (pid=94312)
  Write of size 8 at 0x7b0400000080 by thread T2:
    #0 std::unordered_map<...>::operator[] src/storage_engine.cpp:6
    #1 StorageEngine::Put(...) src/storage_engine.cpp:5
    ...
  Previous read of size 8 at 0x7b0400000080 by thread T1:
    #0 std::unordered_map<...>::find(...) src/storage_engine.cpp:11
    #1 StorageEngine::Get(...) src/storage_engine.cpp:10
    ...
=====
```

Once this test completes with a clean bill of health under ThreadSanitizer, your core data layer is mathematically verified as safe, and you are ready to wrap it inside a gRPC server network interface.

User prompt: how to create shared_mutex object in c++? getting this error: namespace "std" has no member "shared_mutex" C/C++(135) I checked this: gcc --version Apple clang version 21.0.0 (clang-2100.1.1.101)

Response: This error occurs because `std::shared_mutex` was introduced in **C++17**. The error code `C/C++(135)` is specifically generated by the VS Code C/C++ extension (IntelliSense) when it tries to find a feature that isn't available in the C++ standard it is currently targeting (which defaults to an older standard like C++11 or C++14).

Since your Apple Clang version completely supports C++17 and later, you can fix this by addressing three things: the code include, the compiler flags, and the VS Code IDE configuration.

Step 1: Ensure the correct header is included

`std::shared_mutex` lives in its own `<shared_mutex>` header, **not** in `<mutex>`. Your code should look like this:

```
#include <shared_mutex>
#include <mutex> // Needed if you use std::unique_lock or std::shared_lock

std::shared_mutex clear_mutex; // Declare the object

void read_data() {
    // Shared lock for readers (multiple threads can read simultaneously)
    std::shared_lock<std::shared_mutex> lock(clear_mutex);
    // ... read operations ...
}

void write_data() {
    // Exclusive lock for writers (only one thread can write)
    // unique_lock is defined in <mutex>
    std::unique_lock<std::shared_mutex> lock(clear_mutex);
    // ... write operations ...
}
```

Step 2: Fix the VS Code IntelliSense Error C/C++(135)

To stop VS Code from showing the red squiggly line, you must tell its IntelliSense engine to look at C++17 or higher.

1. Press `Cmd + Shift + P` (on Mac) to open the Command Palette.
2. Search for and select **C/C++: Edit Configurations (UI)**.
3. Scroll down to the **C++ standard** section.
4. Change it from the default to **c++17** (or **c++20**).

Alternatively, if you prefer editing configuration files directly, open `.vscode/c_cpp_properties.json` in your project root and update or add the `cppStandard` field:

```
{
  "configurations": [
    {
      "name": "Mac",
      "compilerPath": "/usr/bin/clang++",
      "cStandard": "c17",
      "cppStandard": "c++17", // <-- Change this line
    }
  ]
}
```

```

        "intelliSenseMode": "macos-clang-arm64"
    }
},
"version": 4
}

```

Step 3: Compile with the C++17 Flag

When compiling from your terminal, Apple Clang requires you to explicitly append the standard flag, otherwise, it might fall back to an older base profile.

Via Terminal:

```
clang++ -std=c++17 main.cpp -o main
```

Via CMake (if your project uses it): If you are integrating this into a project build system like CMake, make sure your `CMakeLists.txt` enforces the standard directly:

```

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

```

User prompt: ok, got it, just changing the c++ version in c++ standard section of >C/C++ Edit Configurations (UI) solved the error. Now another doubt: while using shared_mutex, should i use the same function name while using the unique_lock and shared_locks? like unique_lock lock(mutex_) and shared_lock lock(mutex_) or unique_lock lock1(mutex_) and shared_lock lock2(mutex_) or also unique_lock lock(mutex_1) and shared_lock lock(mutex_2)

Response: Awesome! Glad the IntelliSense configuration change got rid of those red squiggles.

To answer your new question, your confusion comes from mixing up two completely different things: the **Mutex Object** (the actual guard) and the **Lock Variable Name** (the local wrapper).

Here is exactly how you should handle both parts for your key-value store.

1. The Mutex Object: Always Use the Exact Same Instance

To protect your internal `std::unordered_map` (or whatever storage engine you're using), both readers and writers **must point to the exact same physical mutex object**.

- **Correct:** Passing `mutex_` to both locks.
- **Dangerous / Broken:** Using `mutex_1` for writing and `mutex_2` for reading. If they are different mutexes, they operate completely independently. A writer locking `mutex_1` will **not** block a reader locking `mutex_2`, completely defeating the purpose and causing data races.

2. The Lock Variable Names: Use `lock` Everywhere

The names you give to `std::unique_lock` and `std::shared_lock` are just local variable names. Because your write operations (like `Put`) and your read operations (like `Get`) are entirely separate functions, their variables exist in **completely different scopes**.

Therefore, you can—and should—name them both `lock`. It keeps your code clean and readable.

What This Looks Like in Your Code

Here is the cleanest, industry-standard way to structure your class:

```

#include <string>
#include <unordered_map>
#include <shared_mutex>
#include <mutex>

class KeyValueStore {
private:
    std::unordered_map<std::string, std::string> db_;
    std::shared_mutex mutex_; // The ONE mutex to rule them all

public:
    // WRITER: Needs exclusive access
    void Put(const std::string& key, const std::string& value) {
        // Automatically locks exclusively on construction, unlocks on exit
        std::unique_lock lock(mutex_);
        db_[key] = value;
    }

    // READER: Can share access with other readers
    std::string Get(const std::string& key) {
        // Automatically takes a shared lock, multiple readers can run this at once
        std::shared_lock lock(mutex_);
        if (db_.find(key) != db_.end()) {
            return db_[key];
        }
    }
}

```

```
    return "";  
}  
};
```

Quick Breakdown of Your Scenarios:

- `unique_lock lock(mutex_)` and `shared_lock lock(mutex_)` → **Perfect**. This is the cleanest way. Because they are in different functions, the names don't clash.
 - `unique_lock lock1(mutex_)` and `shared_lock lock2(mutex_)` → **Compiles, but redundant**. You only need unique names like `lock1` and `lock2` if you are creating two locks *inside the exact same function block*, which you won't be doing here.
 - `unique_lock lock(mutex_1)` and `shared_lock lock(mutex_2)` → **Fatal Error**. This will break your thread safety because the reader and writer are looking at two different stoplights.
-